Preface

Welcome to "Conversational SQL: Advanced Techniques in NLP-Driven PostgreSQL Data Querying." This project represents a significant step forward in the field of Natural Language Processing (NLP) and database management, showcasing how the power of OpenAI's language models can be harnessed to simplify and enhance the process of querying data stored in a PostgreSQL database.

In this project, we have designed and implemented a system that transforms natural language prompts into accurate SQL queries, allowing users to interact with their data in a more intuitive and efficient manner. The system is built using the Faker library to generate a comprehensive sales order schema database, which includes the following tables: Customer Table, Employee Table, Product Table, Supplier Table, Sales Order Table, Sales Order Line Item Table, and Inventory Table. These tables have been populated with realistic data and transferred to a PostgreSQL database. To facilitate user interaction, we have developed an interface using Streamlit. This interface takes user prompts, processes them using advanced prompt engineering techniques, and generates the corresponding SQL queries. The OpenAI model is leveraged to interpret the user's intent and provide accurate, context-aware responses.

Our database schema is designed to support a wide range of sales order queries. The primary tables include the Customer Table, containing customer details such as customer ID, name, contact information, and address; the Employee Table, which records information about employees, including employee ID, name, department, and salary; the Product Table, listing all products with details like product ID, name, description, and price; and the Sales Order Table, capturing order information, including order ID, customer ID, order date, and status. Each table in the schema has been meticulously described to include column names, data types, and any special constraints or keys. This detailed documentation ensures that queries can be constructed accurately and efficiently.

We provide instructions for analyzing the user's query to capture the intent and identify relevant data points, involving understanding the natural language prompt and mapping it to the corresponding SQL components. Directions are provided to help map the captured intent and data points into accurate SQL syntax, including tips on writing efficient SQL queries and handling complex joins and conditions. To ensure optimal performance, we offer tips on optimizing the generated SQL queries, such as suggestions for using indexes, choosing appropriate join types, and other best practices.

Guidelines are provided for dealing with ambiguous or incomplete queries by asking clarifying questions or making educated guesses to ensure the generated SQL query meets the user's needs. Advice is given on formatting query results for easy interpretation by the end user, including organizing data in a readable format and highlighting key information. We emphasize the importance of sanitizing inputs to prevent SQL injection attacks and other security vulnerabilities, highlighting best practices for secure coding to protect the integrity of the database.

Example queries and their corresponding SQL translations are provided for reference, demonstrating how natural language prompts can be converted into accurate SQL queries and the